COMP 551 – McGill, Winter 2026

Topic 13: Attention Mechanism & Transformers

Lecturers: Siamak Ravanbakhsh & Pascale Gourdeau *Notes: Raphaël Michnik*

Certification: I wrote these notes myself, without any LLMs

1 Attention Mechanism

1.1 Motivation: Models for Sequence Data

When processing sequence data, different architectural families use different inductive biases to capture patterns. Convolutional Neural Networks (CNNs) rely on translation invariance and locality, computing outputs as a weighted sum of neighboring input units. Recurrent Neural Networks (RNNs) update a hidden state sequentially through time to summarize past observations. However, this creates a bottleneck in standard encoder-decoder architectures, because the whole input sequence has to be squeezed into a single context vector. Transformers avoid this by replacing recurrence with attention. Instead of passing information step by step through time, each position can look directly at the parts of the input that matter most. This helps the model capture long-range relationships and makes parallel computation possible.

1.2 Idea

When answering certain queries we should focus on relevant parts of data. In an image, that might mean looking at the region containing the object mentioned in a caption. More generally, for sequences or sets of elements, attention lets the model select and combine the most useful information to produce and output.

Figure 1: Example of attention in image captioning. When generating the word "dog", the model can focus on the region of the image containing the dog, rather than processing the entire image uniformly.

Different parts of the input can have different importance depending on the query. The output can be generated by focusing on and combining the most relevant elements.

1.3 Attention Mechanism

Objective: Given a set of inputs vectors $x_1, \ldots, x_T$ and a query vector q, create an output based on the query and the input.

Each input vector x_i is compared with the query to determine its relevance. These relecance scores are converted into weights, and the output is formed as a weighted combination of the input vectors.

For each input x_t we construct a **key** k_t and a **value** v_t. The key is used to determine how relevant that input is to the query, and the vlaue is the information that will contribute to the output. We might use $v_t = k_t = x_t$ or transformed versions $v_t = W_v x_t$ and $k_t = W_k x_t$.

First compare the query q with each key k_t using a scoring function, then apply a softmax functno so taht the scores become normalized attention weights α_t that sum to 1. Using softmax:

$$\alpha_t(q, k_1, \ldots, k_T) = \mathcal{S}(q^T k_1, \ldots, q^T k_T)$$

The output is a weighted sum of values:

$$\text{Attn}(q, (k_1, v_1), \ldots, (k_T, v_T)) = \sum_{t=1}^{T} \alpha_t v_t$$

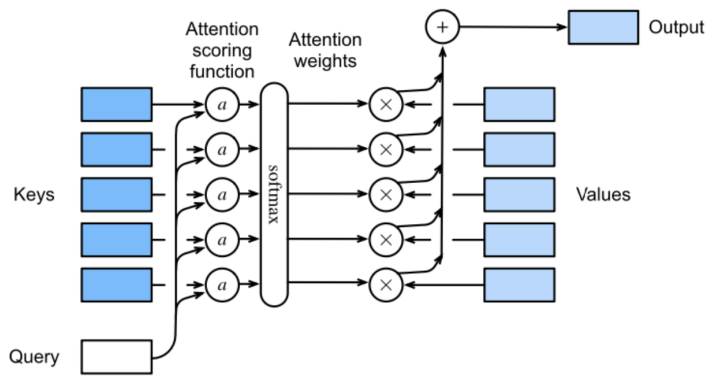

Figure 2: Diagram of the attention mechanism. The query is compared with all keys to produce attention weights, which are then used to form a weighted sum of the value vectors.

There are many choices for the scoring function $a(q, k)$. The simplest is **dot-product attention:** $a(q, k) = q^T \cdot k$, assuming $q, k \in \mathbb{R}^D$. A common improvement is **scaled dot-product attention:** $\frac{q^T \cdot k}{\sqrt{D}}$, which doesn't scale with D. Another option is **additive attention:** $a(q, k) = w^T \cdot \tanh(W_q q + W_k k)$.

If all inputs are stacked into a matrix X, we can compute keys and values as

$$\underset{T \times D_k}{K} = \underset{T \times D}{X} \underset{D \times D_k}{W_k} \qquad \underset{T \times D_v}{V} = \underset{T \times D}{X} \underset{D \times D_v}{W_v}$$

$$\text{Attn}(q, K, V) = \underset{D_v}{\mathcal{S}(qK^T)V}$$

Example: SeqToSeq with Attention: Recall that the the context vector in encoder-decoder RNN is a "bottleneck", as all information must pass through one vector and long-range dependencies are hard to preserve. In tasks like translation, different output words depend on different parts of the input.

When generating each output word, the model should focus on different input words. Without attention, it cannot do this explicitly.

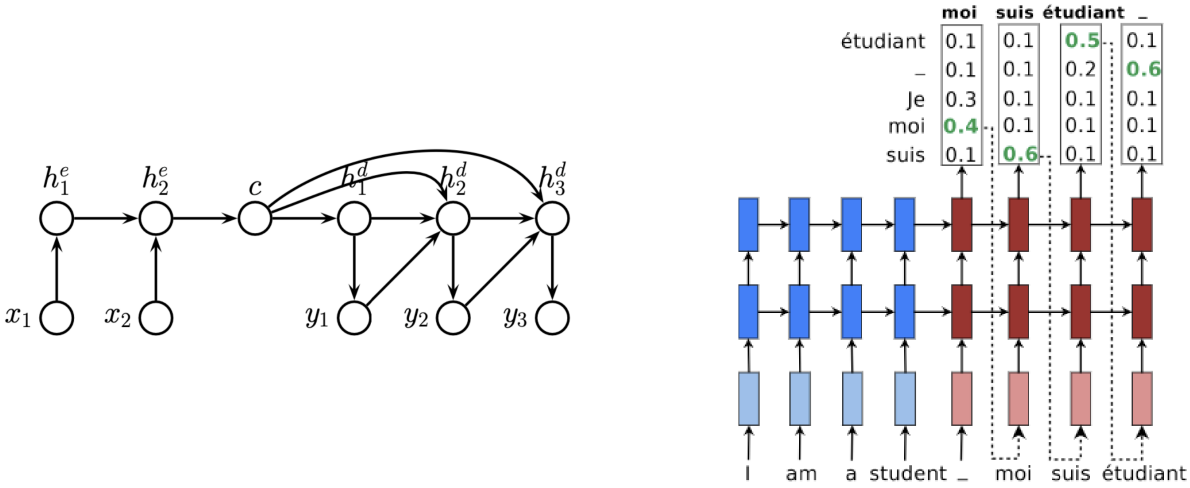

Figure 3: Neural Machine Translation without and with attention. The left diagram shows a standard encoder-decoder architecture where the encoder produces a single context vector that the decoder uses to generate the output sequence. The right diagram shows how attention allows the decoder to focus on different parts of the input sequence when generating each output word, improving performance

Attention replaces the fixed context vector with a dynamic one that is recomputed at each decoding step. Instead of relying on a single summary, the decoder selectively focuses on different encoder states depending on what it is generating. This allows the model to align input and output elements and capture long-range dependencies more effectively.

At each decoding step, we compute a context vector:

$$c_t = \sum_{i=1}^{T} \alpha_i(h_t^d, h_i^e) h_i^e$$

We compare the current decoder hidden state h_t^d with each encoder hidden state h_i^e to get attention weights α_i. Then take a weighted sum of encoder states. The output depends on the decoder state and the context vector, the model is effectively "looking back" at the input each time it generates a word.

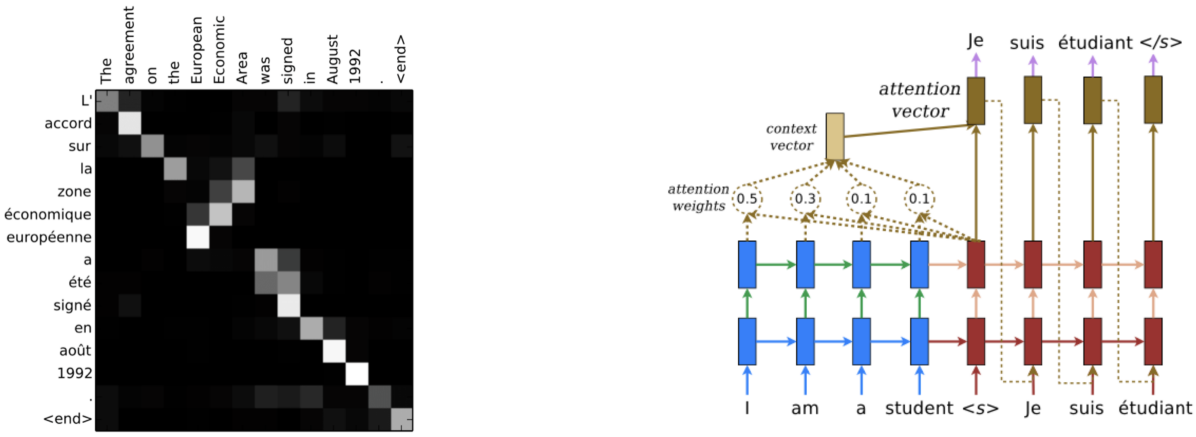

Figure 4: Attention heatmap and output. The heatmap shows which input words are used for each output word. At each step, the decoder computes a context vector by attending to all encoder states. This reveals an alignment between source and target sequences.

1.4 Self-Attention

Self-attention layer is the core building block in transformers. Given an input set/sequence of vectors, we now generate an output sequence/set of vectors.

Given input $x_1, \ldots, x_T$, we compute query $q_t = W_q x_t$, key $k_t = W_k x_t$, and value $v_t = W_v x_t$ value for each. We then generate each outputs $y_1, \ldots, y_T$ using the attention mechanism:

$$y_t = \text{Attn}(q_t, (k_1, v_1), \ldots, (k_T, v_T))$$

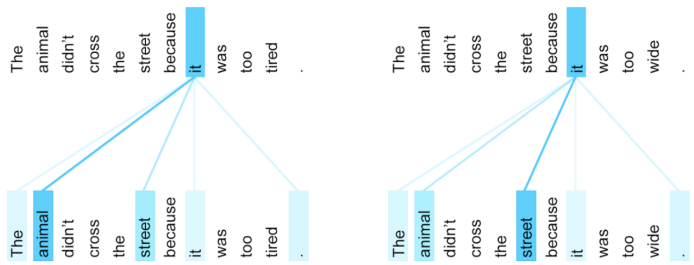

Figure 5: Self attention in transformers. The same word "it" can attend to different words. Each output, pays attention to each input to varying degree we can learn the weights to pay attention to what is relevant for each outpur

We can use **dot product attention** for the entire input:

$$\begin{bmatrix} - & y_1^\top & - \\ & \vdots & \\ - & y_T^\top & - \end{bmatrix} = \mathcal{S}\left(\begin{bmatrix} - & q_1^\top & - \\ & \vdots & \\ - & q_T^\top & - \end{bmatrix} \begin{bmatrix} | & & | \\ k_1, & \ldots, & k_T \\ | & & | \end{bmatrix} \right) \begin{bmatrix} - & v_1^\top & - \\ & \vdots & \\ - & v_T^\top & - \end{bmatrix}$$

T x T attention matrix

Each row sums to one. Each row of the attention matrix gives a linear combination of rows of V. We can use an **attention mask** to prevent some outputs from attenting parts of input. For example a **causal mask** prevents looking into the future in sequence generation.

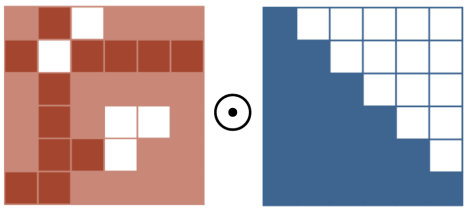

Figure 6: Attention mask. We can optionally multiply the attention matrix by a mask matrix so that predictions don't look into the future

We often use **scaled dot-product attention** in practice. The short matrix form of self-attention:

$$XW_q \quad XW_k \quad XW_v$$

$$Y = \mathcal{S}\left(\frac{QK^\top}{\sqrt{D}}\right)V$$

1.5 Multi-headed Attention

Multiple attention "heads" can detect different types of relationships/patterns. The outputs of these heads are then combined, allowing the model to capture richer patterns than a single attention mechanism. They use different attention weights:

$$y_t^{(h)} = Attn(W_q^{(h)} x_t, \{W_k^{(h)} x_{t'}, W_v^{(h)} x_{t'})\}_{t'=1}^{T})$$

Because projections are different, each head can focus on different positions, capture different types of dependencies, and specialize. Outputs are concatenated and projected with a matrix $W_o$to get the final output. This is similar to channels in CNNs.

$$y_t = MHAttn(q_t, \{(k_{t'}, v_{t'})\}_{t'}) = W_o \begin{bmatrix} y_t^{(1)\top} \\ \vdots \\ y_t^{(H)\top} \end{bmatrix}$$

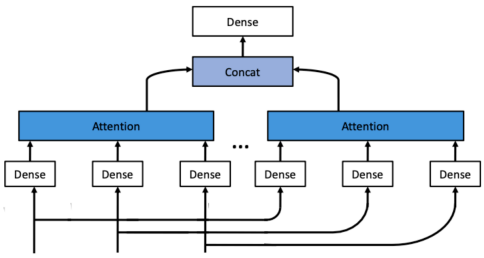

Figure 7: Multi-head attention. Multiple attention heads learn different relationships in parallel, and their outputs are combined to form a richer representation.

1.6 Tokenization and embedding

Before a model can process text, it must convert it into a numerical form. This is done in two steps: first, the text is split into tokens using a tokenization algorithm, and then each token is mapped to a vector through an embedding. This allows the model to work with discrete language inputs using continuous representations.

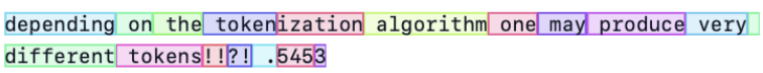

Figure 8: Tokenization. Text is split into discrete units.

Vocabulary consists of all possible tokens, usually including a special $< unknown >$ token for unknown words. Each token becomes a vector of length |vocabulary| assuming **one-hot encoding**: one entry is 1 and the rest are 0.

We map tokens to dense vectors using an embedding matrix: $x_t \leftarrow W_{\text{embed}}\, x_t$

Self-attention treats inputs as a set and does not encode the order of elements. TO allow the model to reason about sequences, we inject positional information into the input with positional encodings, which provide each token with information about its position in the sequence.

- **Idea 1:** Raw position. Just append the index t, $x_t \leftarrow \begin{bmatrix} | \\ x_t \\ | \\ t \end{bmatrix}$. Simple but not very expressive.

- **Idea 2:** Binary encoding. Represent the position in binary, different bits capture different scales $x_2 \leftarrow \begin{bmatrix} | \\ x_2 \\ | \\ 0 \\ 1 \\ 0 \end{bmatrix}$, $x_3 \leftarrow \begin{bmatrix} | \\ x_3 \\ | \\ 0 \\ 1 \\ 0 \end{bmatrix}$.

- **Idea 3:** Sinusoidal encoding. Use sine and cosine functions of different frequencies to encode positions, allowing the model to learn relative positions and generalize to longer sequences.

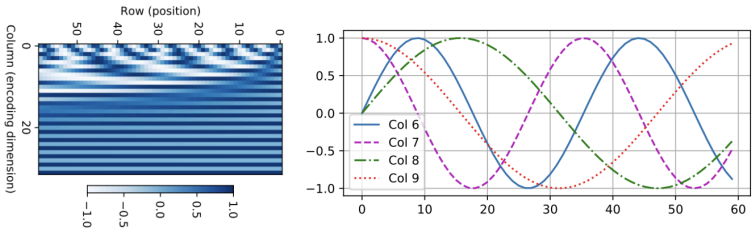

Figure 9: Positional Encoding (PoE).Different frequencies encode position, enabling the model to recover sequence order.

Sinusoidal positional encoding is preferred over binary because it gives positions a smooth, structured representation that reflects both order and distance. Nearby positions get similar vectors, and shifts in position create predictable changes, which helps models learn relative relationships between tokens. Binary encoding changes abruptly and does not preserve positional closeness, so positions that are numerically close can look very different. Sinusoidal encoding is the one actually used in transformers.

2 Transformers

The Transformer combines the ideas introduced so far into a single architecture for processing sequences. Input tokens are first converted into embeddings and enriched with positional information, then passed through a stack of identical blocks. Each block uses multi-head self-attention to exchange information across the sequence and a feedforward network to further transform each position, with residual connections and layer normalization helping optimization and stability.

2.1 Architecture

Inside one block, we first have multi-head self-attention, each position can attend to all other psotions, this lets information flow across the whole sequence. Then comes a feedforward network, which is applied independently to each position, allowing for complex transformations. Around both sublayers we use residual connections which add the input back to the output, and layer normalization which keeps activations well-scaled. These make deep transformers easier to train.

```
def TransformerBlock(X):
    Z = LayerNorm(MultiheadAttn(X) + X)
    E = LayerNorm(FeedForward(Z) + Z)
    return E


def Transformer(X):
    E = POS(Embed(X))
    for l in range(L):
        E = TransformerBlock(E)
    return E
```

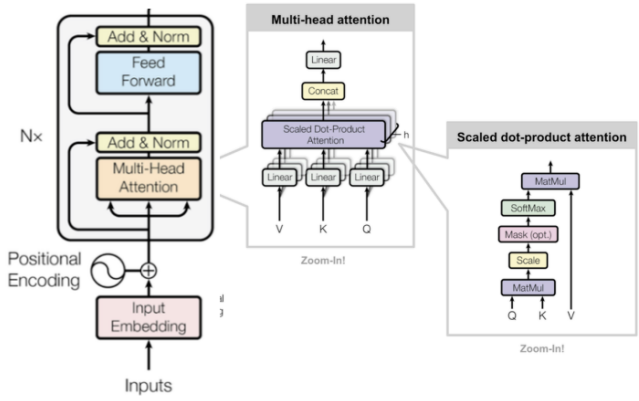

Figure 10: A Transformer alternates sequence-wide information mixing through self-attention with position-wise processing through feedforward layers, repeated across multiple blocks.

2.2 Bidirectional Encoder Representations from Transformers (BERT)

BERT is a Transformer-based model that learns contextual word representations using a bidirectional encoder. Unlike generative models, it is trained to understand text by looking at both left and right context simultaneously. This is achieved through pretraining objectives that encourage the model to recover missing information and understand relationships between sentences.

Pretraining Objectives: Masked Language Modeling (MLM) randomly masks some input tokens and trains the model to predict them based on the surrounding context forcing the model to understand context deeply.

Next Sentence Prediction (NSP) trains the model to determine if two sentences are consecutive in the original text, helping it learn relationships between sentences.

The model is pretrained on a large dataset with these objectives. It learns general-purpose language representations.

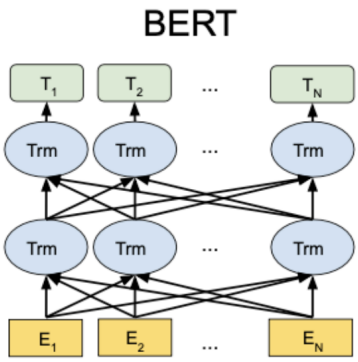

Figure 11: BERT uses a bidirectional Transformer encoder trained with masked token prediction and sentence-level objectives.

After pretraining, BERT can be adapted (fine-tuned) to many downstream tasks by adding a small task-specific layer and fine-tuning the entire model. This allows it to leverage its learned representations with relatively little labeled data.

Finetuning: Start from pretrained BERT and add a small task-specific head (e.g., classifier). Train on labeled data for the target task, this usually requires much less data than training from scratch. Pretraining learns syntax, semantics, contextual relationships, and finetuning specializes these representations.

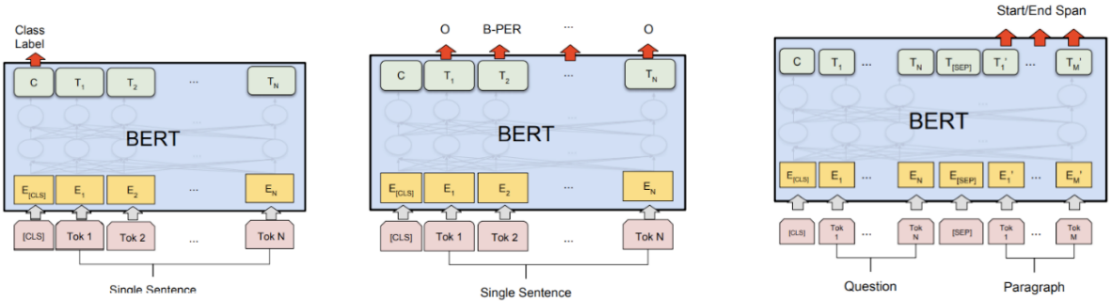

Figure 12: The same pretrained BERT model can be adapted to different tasks by adding simple task-specific heads: Sentence classification, token level tasks, question answering

2.3 Generative Pretrained Transformer (GPT)

GPT is a Transformer-based model designed for text generation. Unlike BERT, it uses a decoder architecture with causal masking, meaning each token can only attend to previous tokens. It is trained to predict the next token in a sequence, allowing it to generate text autoregressively.

Training objective: Given tokens $x_1, \ldots, x_{t-1}$, train the model to predict x_t. Training in parallel is possible by using a causal mask to prevent looking at future tokens: input $x_1, x_2, \ldots, x_T$, targets $x_2, x_3, \ldots, x_{T+1}$ This is just a shifted vesion of the input, and the model learns to predict the next token based on all previous tokens, similar to sequence modeling with RNNs but without the sequential bottleneck.

The model learns: grammar, structure, and long-range dependencies. By predicting the next token, it implicitly models the full sequence distribution.

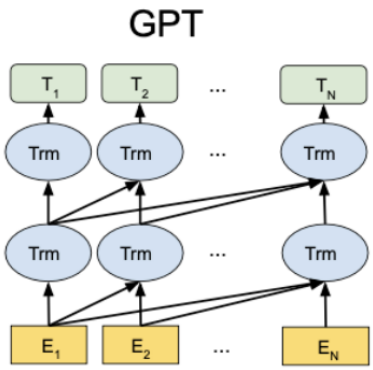

Figure 13: GPT uses a Transformer decoder with causal masking to predict the next token based on previous ones.

At inference: generate one token at a time, append it to the sequence, and repeat. This allows for flexible generation of text of arbitrary length. The model can generate coherent paragraphs, answer questions, or even write code by leveraging its learned language understanding. BERT reads the whole sentence to understand it; GPT reads left-to-right to generate it.

Comparison of three families of model for sequence data

Layer type	Complexity	Sequential ops.	Max. path length
Self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\frac{n}{k})$

n=T: sequence length
d: number of features
k: convolution kernel size

Max. path length connecting any two elements in the sequence

Figure 14: Self-attention enables direct connections between all tokens, reducing dependency path length and enabling parallel computation.

Different neural architectures capture dependencies in sequence data using different inductive biases. Convolutional networks focus on local patterns, recurrent networks process sequences step by step, and Transformers use self-attention to directly connect all elements. These differences lead to trade-offs in computational cost, parallelization, and how easily long-range dependencies can be modeled.

Optional: The lecture slides contain some optional material on BERT and GPT, mostly diagrams and examples.